

Spęczator 2000

Szymon Miłkowski

12.01.2026

1 Cel i wstępne założenia

Celem tego projektu było stworzenie graficznego programu komputerowego do analizy, korekcji i transformacji danych pomiarowych z prób spęczania. Danymi wejściowymi do programu miały być pliki binarne w formacie *W01* generowane przez oprogramowanie maszyny badawczej.

Dodatkowym celem miała być kompatybilność i płynność działania zarówno na współczesnym sprzęcie, jak i starszym (Windows XP 32-bit).

2 Wybór technologii

Z racji na wymóg szerokiej kompatybilności, oraz wysokiej wydajności, konieczne było wybranie technologii pozwalającej na kompilację do natywnego kodu maszynowego z minimalną ilością zależności. Wybór padł na język C z biblioteką Raylib do obsługi grafiki, linkowaną statycznie.

Raylib posiada łatwe w obsłudze API składające się z funkcji do rysowania i obsługi wejścia (mysz, klawiatura).

3 Spęczanie

Próba spęczania polega na ściskaniu próbki wzdłuż jednej osi (pionowej) przez maszynę badawczą z zadaną prędkością. Podczas próby materiał odkształca się, zmniejszając swoją wysokość w kierunku siły oraz - zgodnie z zasadą zachowania objętości - zwiększając swoją szerokość w pozostałych kierunkach.

Podczas tego procesu, materiał wewnątrz płynie, wypychając materiał na ścianach bocznych na zewnątrz, a materiał przy podstawach trze o górną i dolną ścianę maszyny, co hamuje jego przemieszczenie na zewnątrz.

Siła nacisku (opór jaki stawia próbka) jest mierzona i zależy od materiału i fazy procesu.

3.1 Fazy procesu

Brak kontaktu

- Brak kontaktu - maszyna nie ściska próbki, zmierza w jej kierunku - stała, niska siła
- Pierwszy kontakt - luzy w maszynie są ściskane - szum informacyjny
- Faza liniowa?? - siła rośnie liniowo
- Faza nieliniowa?? - siła rośnie coraz mniej

4 Format pliku wejściowego

5 Implementacja

5.1 Ogólna struktura aplikacji

5.1.1 Komponenty

Interfejs graficzny aplikacji podzielony jest na "komponenty" - funkcje zawierające logikę kontrolną oraz rysujące pewien element interfejsu. Wywołanie tej funkcji oznacza, że komponent istnieje w obecnej klatce. Funkcje te mogą wywoływać inne funkcje komponentów, co tworzy swego rodzaju strukturę drzewa komponentów, rozpoczynającą się w głównej pętli programu. Pozycja w strukturze drzewa nie ma jednak wpływu na rozmieszczenie elementu, pozycja jest statyczna, lub określana poprzez podane parametry.

5.1.2 Zarządzanie stanem

Inspirowane działaniem innego drzewiastego systemu komponentów graficznych - webowym Reactem - komponenty, jeśli to konieczne, operują na dwóch rodzajach danych: właściwości (*props*) i stan (*state*).

Właściwości są podawane do funkcji komponentu jako kopia (pass by value), bądź stały wskaźnik, mają za zadanie sterować zachowaniem i wyglądem komponentu.

Stan jest podawany jako wskaźnik do pamięci zarządzanej przez rodzica (komponent wywołujący), ma za zadanie przechowywanie stanu komponentu między wywołaniami, jest modyfikowany głównie przez sam komponent oraz jego dzieci (jeśli zostanie podany do nich jako ich stan).

Stan globalny jest wykorzystywany w niektórych komponentach, które z założenia mają istnieć tylko w jednej sztuce jednocześnie, jak lista otwartych plików lub narzędzia diagnostyczne *FPSCounter* i *Clock*, jako zmienne globalne w jednostce translacyjnej (niewidoczno poza obecnym plikiem .c), lub statyczne zmienne lokalne w funkcji komponentu.

5.2 Rysowanie wykresu